

1-2 · 쉘 스크립트 · 이론 1H

쉘 스크립트 기초 문법

변수·인자·조건·반복·종료 코드 — 한 시간에 훑기

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1시간 뒤에는

- ① 서버·변수·위치 인자·종료 코드의 역할을 말할 수 있다
- ② if 와 파일 테스트 연산자로 갈라지는 스크립트를 읽는다
- ③ 명령 치환 \$() 으로 명령 결과를 변수에 담는다
- ④ [가 명령이라 대괄호 안쪽에 공백이 필요한 이유를 안다

다음 1.5H 실습에서 이 문법으로 인사·검사·집계 스크립트를 직접 만듭니다

1. 왜 스크립트인가

손으로 치는 명령의 세 가지 문제

1 기억나지 않는다
어제 무엇을 쳤는지 되짚을 수 없다

2 남에게 시킬 수 없다
똑같이 해 달라고 넘길 방법이 없다

3 새벽 3시에는 아무도 못 친다
사람이 없으면 돌지 않는다

4 스크립트 = 요리 레시피
순서를 적어 두면 언제 누가 실행해도 같은 결과 — 데이터 파이프라인의 재현성이 여기서 시작

5 그리고 Git 에 올라간다
처리 로직의 변경 이력이 남는다

첫 줄 셸뱅 · 등호 양옆 공백 금지 · 쓸 때는 "\$변수"

#!/bin/bash

01

「이 레시피는 bash 가 읽는다」는 표시
./파일명 으로 직접 실행하려면 필수

name="데이터"

02

등호 양옆에 공백 없이 선언
name = "데이터" 는 command not
found

"\$name"

03

앞에 \$ · 항상 큰따옴표로 감싼다
\${count}개 — 중괄호는 이름의 경계

따옴표가 없으면 값에 공백이 있을 때 여러 개로 쪼개진다 — 파일 이름에 공백이 든 경우 스크립트가 깨지는 가장 흔한 이유

3. 위치 인자

실행할 때 뒤에 붙인 값이 \$1 \$2 \$3 에 순서대로

표기	뜻
\$0	스크립트 자기 이름
\$1 \$2 \$3 ...	첫째 · 둘째 · 셋째 인자
\$#	넘어온 인자의 개수
\$@	모든 인자를 하나씩 (반복문에서)
\$?	직전 명령의 종료 코드

./greet.sh 홍길동 → \$1 에 홍길동이 담긴다 · \$? 는 직전 명령의 종료 코드

4. 명령 치환

\$() — 괄호 안 명령의 결과가 그 자리에 들어온다

1 `today=$(date +%Y%m%d)`
+ 뒤는 「어떤 모양으로 찍을지」 — %Y 연도 네 자리 · %m 월 · %d 일 → 20260922

2 `lines=$(wc -l < access.log)`
파일 줄 수를 변수에 담기 — echo "줄 수: \$lines"

3 백틱 `명령` 은 옛 문법
같은 뜻이지만 \$() 를 쓴다 — 중첩이 된다

%Y%m%d

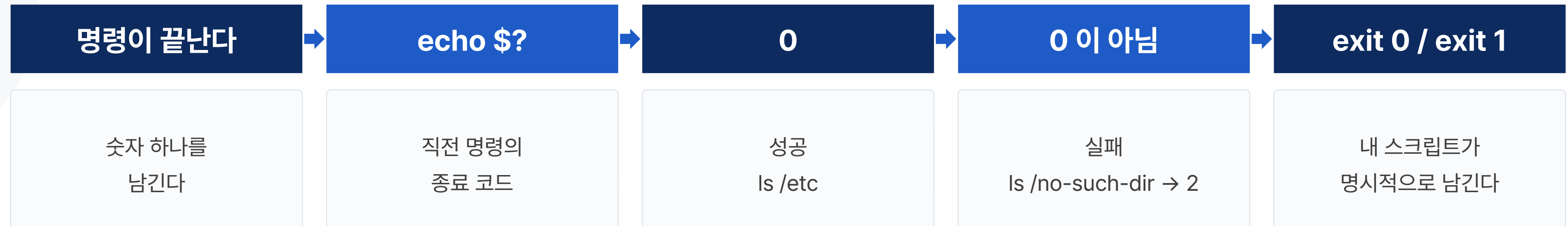
20260922

연.월.일

%가 아닌 글자는 그대로 찍힌다 · %F 는 %Y-%m-%d — 파일 이름에 쓸 때는 공백과 콜론(:)을 피한다

5. 종료 코드

자동화가 보는 유일한 신호 — 0 이면 성공, 아니면 실패



실패 코드는 명령마다 다를 수 있다 — 약속은 「0 = 성공」 하나

사람은 화면을 보지만, 새벽에 자동으로 도는 작업은 이 숫자가 성패 판단의 전부

6. 조건문

if [조건]; then ... fi — 파일 테스트 연산자

연산자	참이 되는 조건
-f 경로	그 자리에 일반 파일이 있다
-d 경로	그 자리에 디렉터리가 있다
-e 경로	파일이든 폴더든 존재한다
-s 경로	파일이 있고 크기가 0 보다 크다
-z "\$var"	변수가 비어 있다
-n "\$var"	변수에 값이 있다
"\$a" = "\$b"	두 문자열이 같다
\$a -eq \$b	두 숫자가 같다 (-lt -gt 계열)

[는 문법 기호가 아니라 하나의 명령 — [뒤와] 앞에 공백이 없으면 command not found

닫는 말은 여는 말을 거꾸로 쓴다

여는 말 · 닫는 말

빠뜨리면 syntax error: unexpected end of file — 끝을 알릴 낱말이 따로 있다

여는 말	닫는 말	비고
if	fi	거꾸로 쓴 것
case	esac	역시 거꾸로
for · while	done	사이에 do 를 둔다

VS

반복문 세 모양

목록 · 숫자 · 파일을 한 줄씩

- for f in *.log; do ... done
- for i in 1 2 3; do ... done
- while read -r line; do ... done < access.log — 실무에서 아주 자주
- then 앞의 세미콜론 — 줄을 바꾸면 필요 없다

이 1시간에서 꼭 가져갈 것

- 1 첫 줄 셔뱅이 어느 셀로 읽을지 정한다
- 2 변수는 등호 양옆에 공백 없이 선언하고, 쓸 때는 항상 "\$변수"로 감싼다
- 3 \$1 은 첫 번째 인자, \$? 는 직전 명령의 종료 코드다
- 4 [는 명령이므로 대괄호 안쪽에 공백이 필요하다
- 5 자동화는 화면 메시지가 아니라 종료 코드를 본다

1-2 · 이론 1H 마무리

다음은 스크립트 작성 실습 (1.5H)

인자를 받고, 조건으로 갈라지고, 반복하는 스크립트를 직접 만듭니다 — 일부러 한 번 깨뜨려 보는 단계가 들어 있습니다

✓ `greet.sh` — 인자와 사용법 안내

✓ 다음표를 빼고 실행해 쪼개짐을 눈으로 확인

✓ `check_file.sh` · `scan_logs.sh` — 종료 코드와 `for`

✓ `report_tool.sh` — 함수·`case`·`set -euo pipefail`

준비할 것

1-1 의 실습 폴더와 `access.log` 8줄